

Feb 23, 2025 13:05

ControlDB_new.py

Page 1/20

```
1  # this controls the database interactions in pyside6
2  import sys
3  import os
4  import platform
5  from loguru import logger
6
7  import numpy as np #for pyinstaller
8  import config_mycal as CM
9  import IngredientTable as INTA
10
11  from Recipe1 import Ui_MainWindow
12
13  from missing_ingredient import Ui_MissingIngredient
14  from missing_ingredient_dialog import Ui_missing_ingredient_dialog
15  from pick_swiss_ingreds import Ui_Dialog
16  import pandas as PD # to pack the recipe information into a pandas dataframe
17  from IngredientTable import IngredientTable
18
19
20  from PySide6.QtCore import (QSize, Qt ,QRect,
21  QCoreApplication,Slot,Signal,QAbstractListModel,QAbstractTableModel,QMetaObject,QModelIndex)
22  from PySide6.QtGui import QAction,QDoubleValidator,QFont
23
24  from PySide6.QtUiTools import QUiLoader
25
26  from PySide6.QtSql import QSqlDatabase , QSql,QSqlTableModel,QSqlQueryModel,QSqlQuery
27
28  from PySide6.QtWidgets import (
29  QApplication,
30  QCheckBox,
31  QComboBox,
32  QDateEdit,
33  QDateTimeEdit,
34  QDial,
35  QDialog,
36  QDoubleSpinBox,
37  QFileDialog,
38  QFontComboBox,
39  QFormLayout,
40  QHBoxLayout,
41  QLabel,
42  QLCDNumber,
43  QLineEdit,
44  QListView,
45  QMainWindow,
46  QWidget,
47  QProgressBar,
48  QPushButton,
49  QRadioButton,
50  QSlider,
51  QSpinBox,
52  QStatusBar,
53  QTimeEdit,
54  QTableView,
55  QMenuBar,
56  QVBoxLayout,
57  QAbstractItemView,
```

Feb 23, 2025 13:05

ControlDB_new.py

Page 2/20

```

58     QGridLayout,
59     QTextEdit
60
61 )
62
63 class NewWindow(QMainWindow):
64     def __init__(self, Title = None):
65         super().__init__()
66         layout = QVBoxLayout()
67
68         self.label = QLabel("Another Window")
69         if(Title == None):
70             self.setWindowTitle("MyCal")
71         else:
72             self.setWindowTitle(Title)
73
74         layout.addWidget(self.label)
75         self.setLayout(layout)
76
77
78 class RecipeModel(QAbstractTableModel):
79
80     def __init__(self, recipes=None, header_labels=None):
81         super().__init__()
82
83         #When subclassing QAbstractListModel, you must provide implementations of the rowCount() and data() functions.
84         # Well behaved models also provide a headerData() implementation.
85         # this means we will have to define the data and rowCount
86         # see also https://doc.qt.io/qt-6/qabstractlistmodel.html
87
88         self.recipes = recipes or [[]] #recipes will be a 2d array or a nested list
89         self.header_labels = header_labels = ["weight", "unit", "ingredient"]
90     def data(self, index, role):
91         if role == Qt.DisplayRole or Qt.ItemDataRole.DisplayRole or role == Qt.ItemDataRole.EditRole:
92             value = self.recipes[index.row()][index.column()]
93             # check for type
94             if isinstance(value, float):
95
96                 return "%.2f" % value
97
98             if isinstance(value, str):
99                 return "%s" % value
100
101
102             return value
103
104
105     def rowCount(self, parent=QModelIndex()):
106         return len(self.recipes)
107
108     def columnCount(self, parent=QModelIndex()):
109         return len(self.recipes[0])
110
111     def headerData(self, section, orientation, role=Qt.DisplayRole):
112         if role == Qt.DisplayRole and orientation == Qt.Horizontal:
113             return self.header_labels[section]
114

```

Feb 23, 2025 13:05

ControlDB_new.py

Page 3/20

```

115         return None
116
117     def insertRows(self, position, rows=1, parent=QModelIndex(), *args, **kwargs):
118         self.beginInsertRows(parent, position, position + rows - 1)
119         for _ in range(rows):
120             self.recipes.insert(position, [""] * self.columnCount())
121         self.endInsertRows()
122         return True
123
124     def setData(self, index, value, role):
125         if role == Qt.ItemDataRole.EditRole:
126             self.recipes[index.row()][index.column()] = value
127             return True
128         return False
129     def flags(self, index):
130         return Qt.ItemFlag.ItemIsSelectable | Qt.ItemFlag.ItemIsEnabled | Qt.ItemFlag.ItemIsEditable
131
132     def StoreValue(self):
133         return self.recipes
134
135
136 class MyRecipeWindow(QMainWindow, Ui_MainWindow):
137     def __init__(self):
138         super().__init__()
139         self.setupUi(self)
140
141 class MyMissingWindow1(QMainWindow, Ui_MissingIngredient):
142     def __init__(self):
143         super().__init__()
144         self.setupUi(self)
145
146 class MyMissingIngredientDialog(QDialog, Ui_missing_ingredient_dialog):
147     def __init__(self):
148         super().__init__()
149         self.setupUi(self)
150
151     #def accept(self):
152     #    """overwrite the accept function"""
153     #    #print("custom func")
154     #    super().accept()
155
156
157
158
159
160
161
162 class ControlDB_new(QMainWindow):
163
164     def __init__(self, Title=None, db_name=None, db_user=None, db_system=None, db_pwd = None, config_file = None):
165         super().__init__()
166         self.db_name = db_name
167         self.db_user = db_user
168         self.db_system = db_system
169         self.config_file = config_file
170         if db_pwd == None:
171

```

Feb 23, 2025 13:05

ControlDB_new.py

Page 4/20

```

172         temp = '/Users/klein/git/qt_exercises/config/pw.txt'
173         if os.path.exists(temp):
174             with open(temp, 'r') as file:
175                 password = file.read().rstrip()
176                 self.db_pwd = password
177         else:
178             self.db_pwd = db_pwd
179
180         self.init_variables()
181         self.SetupLogger()
182         #self.CreateIngredientsForm()
183         self.SetupConfig()
184         self.ingredients_suppress_columns = self.CM.ingredients_suppress_columns
185
186         self.recipes_suppress_columns = self.CM.recipes_suppress_columns
187         self.swiss_food_suppress_columns = self.CM.swiss_food_suppress_columns
188
189         self.version = '2.0'
190         logger.info('***** \n\n\n')
191
192         logger.info(' This is version %s' % self.version)
193         logger.info('\n\n\n***** \n')
194
195         #instantiate the IngredientTable
196         table_name = '/Users/klein/git/qt_exercises/nutrition_databases/swiss_food.csv'
197         #if we are using the swiss_food table we need to get the IngredienTable class
198         self.swiss_food = False
199         if(self.CM.ingred_table == "swiss_food"):
200             self.swiss_food = True
201             logger.warning("nned work, pass the table")
202             self.InTa = IngredientTable(table_name=self.CM.ingred_table, ingredients = None)
203
204     def SetupConfig(self):
205
206         # get the config filename
207         # here we do a filedialog
208         if(self.config_file == None or not os.path.isfile(self.config_file)):
209             self.config_file, filter = QFileDialog.getOpenFileName(self,
210                 self.tr("Open Config file"), "~", self.tr("*.json"))
211
212
213         logger.info("config file %s" % self.config_file)
214
215
216
217         self.CM = CM.MyConfig(self.config_file)
218         self.log_level = self.CM.log_level
219         self.log_output = self.CM.log_output
220
221
222     def init_variables(self):
223         """ initialize some variable to None """
224
225         self.missing_ingredient = None
226         self.new_recipe_name = None
227
228     def ConnectDataBase(self):

```

Feb 23, 2025 13:05

ControlDB_new.py

Page 5/20

```

229         ''' establish contact to database'''
230
231
232         #instantiate the connection
233         self.mycal_db = QSqlDatabase.addDatabase(self.db_system)
234         self.mycal_db.setHostName("192.168.2.164")
235         self.mycal_db.setDatabaseName(self.db_name)
236         self.mycal_db.setUserName(self.db_user)
237         self.mycal_db.setPassword(self.db_pwd)
238
239
240     #         self.mycal_db.setPassword(self.CM.pwd)
241
242     result = self.mycal_db.open()
243     if(result):
244         logger.info('connection successful')
245         # here we get the connection name
246         # will be needed when we want to close the connection
247         self.connection_name = self.mycal_db.connectionName()
248         logger.info(' database connection name %s' % self.connection_name)
249         #self.ShowTables()
250     else:
251         logger.error('database problem %s' % self.mycal_db.lastError().text())
252         logger.error('connection failed, exiting')
253         sys.exit(0)
254
255     def SetupLogger(self):
256
257
258         #logger.remove(0)
259         #now we add color to the terminal output
260         logger.add(sys.stdout,
261                     colorize = True,format="<green>{time}</green> {function} {line} {level} <level>{message}</level> " ,
262                     level = "DEBUG")
263
264
265
266         fmt = " {time} - {name} - {function} - {line} - {level} - {message}"
267         logger.add('info.log', format = fmt , level = 'INFO',rotation="1 day")
268
269
270         # set the colors of the different levels
271         logger.level("INFO",color = '<black>')
272         logger.level("WARNING",color='<green>')
273         logger.level("ERROR",color='<red>')
274         logger.level("DEBUG",color = '<blue>')
275
276         return
277
278
279
280     def ShowTables(self):
281         '''prints all the tables in the databe'''
282         self.table_list = self.mycal_db.tables(type=QSql.Tables)
283         for k in range(0,len(self.table_list)):
284             print(self.table_list[k])
285

```

Feb 23, 2025 13:05

ControlDB_new.py

Page 6/20

```

286         #Create list box
287         self.mybox = QComboBox()
288         #Insert at index 0 the whole list
289         self.mybox.insertItems(0,self.table_list)
290
291         # Now sort them alphabetically
292         self.sort_mybox()
293
294         # create independent window
295
296         #self.setCentralWidget(self.mybox)
297
298
299         return
300     def sort_mybox(self):
301         items = [self.mybox.itemText(i) for i in range(self.mybox.count())]
302         items.sort()
303         self.mybox.clear()
304         self.mybox.addItems(items)
305
306     def ViewTable(self,table,suppress_columns=[],editmode=True,columnwidth=[],Title = None, showTable = True):
307         self.table_view = QTableView()
308         self.table_view.setSortingEnabled(True)
309         #self.table_view.sortByColumn(2,Qt.AscendingOrder)
310
311         self.model = QSqlTableModel(db = self.mycal_db)
312
313
314
315         self.table_view.setModel(self.model)
316         #prevent editing
317         if(not editmode):
318             self.table_view.setEditTriggers(QAbstractItemView.NoEditTriggers)
319
320         self.model.setTable(table)
321
322         # get rid of columns
323         columns_to_remove = suppress_columns
324         for cn in columns_to_remove:
325             idx = self.model.fieldIndex(cn)
326             self.model.removeColumns(idx, 1)
327
328
329         self.model.select()
330         # change column width
331         for k in range(0,len(columnwidth)):
332             self.table_view.setColumnWidth(k, columnwidth[k])
333
334         self.setMinimumSize(QSize(600, 300))
335
336         self.setCentralWidget(self.table_view)
337
338         self.SetPosition(10,10)
339         if(showTable):
340             self.show()
341         #self.setCentralWidget(table)
342

```

Feb 23, 2025 13:05

ControlDB_new.py

Page 7/20

```

343
344     def SetPosition(self, pos_x, pos_y):
345         self.move(pos_x, pos_y)
346
347     def CreateIngredientsForm1(self):
348         """uses the form layout"""
349
350         #always use self so window will be persistent
351         self.Inform = NewWindow(Title = "Add New Ingredient")
352         self.Inform.setStyleSheet("background-color: yellow;")
353
354         # setup geometry
355         mysize=[400,300]
356         myposit = [800,100]
357
358         self.SizeWindow(self.Inform,myposit,mysize)
359
360         # Create widgets
361         #fill with ingredients if called from missing ingredients
362         if( self.missing_ingredient != None or self.missing_ingredient != ''):
363             self.name_label = QLineEdit(self.missing_ingredient)
364         else:
365             self.name_label =QLineEdit()
366             self.name_label.setStyleSheet("background-color: rgb(3, 252, 227)")
367             # force valid entr
368             double_validator = QDoubleValidator(0, 500.0, 2)
369
370             self.calory_label = QLineEdit()
371             self.calory_label.setStyleSheet("background-color: white")
372             self.calory_label.setValidator(double_validator)
373
374             self.carb_label = QLineEdit()
375             self.carb_label.setStyleSheet("background-color: white")
376             self.carb_label.setValidator(double_validator)
377             self.fat_label = QLineEdit()
378             self.fat_label.setStyleSheet("background-color: white")
379             self.fat_label.setValidator(double_validator)
380             self.prot_label = QLineEdit()
381             self.prot_label.setStyleSheet("background-color: white")
382             self.prot_label.setValidator(double_validator)
383
384             SaveButton = QPushButton("Save")
385             SaveButton.setStyleSheet("background-color: white")
386             CancelButton = QPushButton("Cancel")
387             CancelButton.setStyleSheet("background-color: red")
388
389         # Create layout
390         form_layout = QFormLayout()
391         form_layout.addRow("Name:", self.name_label)
392         form_layout.addRow("Calories/100g", self.calory_label)
393         form_layout.addRow("Fat", self.fat_label)
394         form_layout.addRow("Carbohydrates", self.carb_label)
395         form_layout.addRow("Protein", self.prot_label)
396         form_layout.addRow(SaveButton)
397         form_layout.addRow(CancelButton)
398
399

```

Feb 23, 2025 13:05

ControlDB_new.py

Page 8/20

```

400     mylayout = QVBoxLayout()
401     mylayout.addLayout(form_layout)
402     self.setLayout(mylayout)
403
404     # Connect submit button
405     SaveButton.clicked.connect(self.SaveIngredients)
406     CancelButton.clicked.connect(lambda : self.Cancel(self.Inform))
407
408     widget = QWidget()
409     widget.setLayout(mylayout)
410
411
412
413
414
415     self.Inform.setCentralWidget(widget)
416
417     #for label in MyLabel:
418     #     widget = QWidget()
419     #     widget.setLayout(layout)
420     #self.setCentralWidget(widget)
421
422
423     # create the form
424
425     self.Inform.show()
426
427
428     def InsertRecord(self,table = None):
429         """ this inserts a record into the cosen table
430         I will use the query function to do this.
431         The record contains the info to be added
432         record=[name,energy,fat,carbohydrate,protein]
433
434         This is the same for ingredients and receipes"""
435
436         #Create table view
437         if (table == "ingredients") :
438
439             self.ViewTable(table = table,suppress_columns=self.ingredients_suppress_columns,showTable= False)
440         elif (table == "recipes") :
441             self.ViewTable(table = table,suppress_columns=self.recipes_suppress_columns)
442         elif (table == "swiss_food") :
443             self.ViewTable(table = table,suppress_columns=self.swiss_food_suppress_columns)
444         else:
445             logger.error("no table given")
446             sys.exit(0)
447
448
449         self.model = QSqlQueryModel()
450         self.table_view.setModel(self.model)
451         #next we check we don't have an entry yet
452
453         #Form the sql INSERT statement
454         #     self.record = [Ing_name,Ing_calory,Ing_carb,Ing_fat,Ing_prot]
455         #INSERT INTO products (product_no, name, price) VALUES (1, 'Cheese', 9.99);
456         #INSERT INTO table (name,energy,carbohydrate,fat,protein) VALUES (self.record[0],

```


Feb 23, 2025 13:05

ControlDB_new.py

Page 9/20

```

457         #                                     self.record[1],
458         #                                     self.record[2],
459         #                                     self.record[3],
460         #                                     self.record[4]);
461         temp = ""+self.record[0]+" "
462         sql = 'INSERT INTO '+table+' (name,energy,carbohydrate,fat,protein) VALUES ('+temp+','+str(self.record[1])+','+str(self.record[2])+','+str(
self.record[3])+','+str(self.record[4])+')';
463         #print(sql)
464
465         #query = QSqlQuery(sql,db=self.mycal_db)
466
467         #self.model.setQuery(query)
468         self.do_sql(sql)
469         logger.info(' inserted record %s into table %s' % (self.record[0],table))
470
471
472
473         return
474
475
476
477     def MyMissingIngredients(self):
478         """ form if ingredient is missing"""
479         if(not self.swiss_food):
480
481             self.MMI1 = MyMissingIngredientDialog()
482             self.MMI1.IngredLine.setText(self.missing_ingredient)
483             self.MMI1.EditButton.clicked.connect(self.show_ingred_form)
484             #self.MMI1.CloseButton.clicked.connect(self.Cancel(self.MMI1))
485
486
487             self.MMI1.move(100,50)
488             t = self.MMI1.exec()
489
490             #if t == QDialog.Accepted:
491             #    self.show_ingred_form()
492             #    #self.MMI1.close()
493
494
495             self.MMI1.show()
496         else:
497             self.InTa.find_pattern(self.missing_ingredient[0:5])
498             self.InTa.get_ingredient()
499             #self.InTa.get_ingred_selection()
500             self.InTa_name = self.InTa.ingredient_name
501             self.InTa_energy = self.InTa.energy
502             self.InTa_carbohydrate = self.InTa.carbohydrate
503             self.InTa_protein = self.InTa.protein
504             self.InTa_fiber = self.InTa.fiber
505             self.InTa_sugar = self.InTa.sugar
506             self.InTa_salt = self.InTa.salt
507             self.InTa_fat = self.InTa.fat
508
509
510         return
511
512

```

Feb 23, 2025 13:05

ControlDB_new.py

Page 10/20

```

513
514     def show_ingred_form(self):
515         # destroy dialog give action back
516         try:
517             self.MMI1.destroy()
518         except:
519             logger.debug("nothing to destroy")
520
521     #         window.CreateIngredientsForm1()
522     self.CreateIngredientsForm1()
523
524
525
526     def MyRecipes_new(self):
527         """using the QTdesigner"""
528         self.MRD = MyRecipeWindow()
529         self.MRD.show()
530
531         # Load the model
532         #self.header_labels = header_labels = ["weight", "unit", "ingredient"]
533
534         self.RecipeModel = RecipeModel()
535         self.MRD.RecipeTableView.setModel(self.RecipeModel)
536
537         # now working self.RecipeModel.setEditStrategy(QSqlTableModel.setEditStrategy.OnFieldChange)
538
539         #label the columns
540         header = self.MRD.RecipeTableView.horizontalHeader()
541
542
543         #poulate the combo box
544         self.MRD.RecipeListCombo.setObjectName(u"RecipeListCombo")
545         self.MRD.RecipeListCombo.addItem(self.GetRecipeList())
546
547
548         #here we define the actions
549         self.MRD.RecipeListCombo.currentTextChanged.connect(self.getSelectedRecipe)
550
551         self.MRD.RecipeTableView.clicked.connect(self.on_item_click)
552
553         # save button
554         self.MRD.SaveRecipe.clicked.connect(self.save_recipe)
555
556         #cancel button
557         self.MRD.CancelButton.clicked.connect(lambda : self.Cancel(self.MRD))
558
559         #close button
560         self.MRD.CloseButton.clicked.connect(lambda : self.Cancel(self.MRD))
561
562         #add row button
563         self.MRD.AddRowButton.clicked.connect(self.add_row)
564
565         #new recipe button
566         #self.MRD.NewRecipeButton.clicked.connect(self.create_new_recipe)
567
568         #if line has change
569         self.MRD.NewRecipeName.returnPressed.connect(self.create_new_recipe)

```

Feb 23, 2025 13:05

ControlDB_new.py

Page 11/20

```

570         #self.MRD.NewRecipeName.textChanged.connect(self.create_new_recipe)
571
572         # here we fill first label
573         self.MRD.version_line.setText(self.version)
574
575     def create_new_recipe(self):
576         """creates new recipe"""
577         #first get name
578         self.new_recipe_name = self.MRD.NewRecipeName.text()
579         logger.info("new recipe name %s" % self.new_recipe_name)
580         #let's create a default starter table
581         #first create the data list for the model
582         self.RecipeModel.recipes = []
583
584         self.recipe_ingredients=[['0','g',''],['0','g',''],['0','g',''],['0','g',''],['0','g','']]
585         self.DisplayRecipeList()
586         self.RecipeModel.layoutChanged.emit()
587
588
589     return
590
591     def create_pandas_frame(self,name=None):
592         """This routine creates the pandas frames which then get passed to CalculateCalories class"""
593         #collect the information
594         logger.info(" name %s and record %s for recipes" % (name,self.panda_ingredients))
595
596         # now we need to get the different quantities for each ingredient
597         # at the same time we need to check if the ingredient is in the list available
598         #if not we have to deal with this
599         number_column = 4
600         energy,protein,carbohydrate,fat = range(number_column) # for referencing the query values
601
602         logger.debug('recipes ingredient: %s' % self.RecipeModel.recipes)
603
604         #example sql statement:
605         # SELECT energy,protein,carbohydrate,fat FROM ingredients where name = 'Butter';
606
607
608         energy_v,protein_v,carbohydrate_v,fat_v = ([ for i in range(number_column)])
609         print(self.RecipeModel.recipes)
610
611         # loop over all ingredients
612         Total_calories_per_gm = 0. # cacluated calories/g
613         Total_protein_per_gm = 0.
614         Total_fat_per_gm = 0.
615         Total_carbs_per_gram = 0.
616         temp_weight = 0.
617         Total_weight = 0.
618         print(self.panda_ingredients)
619         #remove empty rows
620         temp_buf=[]
621         for k in range(len(self.panda_ingredients)):
622             if self.panda_ingredients[k][2] != '':
623                 temp_buf.append(self.panda_ingredients[k])
624
625         self.panda_ingredients= temp_buf
626

```

Feb 23, 2025 13:05

ControlDB_new.py

Page 12/20

```

627     for k in range(len(self.panda_ingredients)):
628         sql = 'SELECT energy,protein,carbohydrate,fat FROM ingredients where name ILIKE \'' + self.panda_ingredients[k][2] + '\';'
629         logger.debug('the query string %s' % sql)
630
631         query = self.do_sql(sql)
632
633         logger.debug('value of query.first %s' % query.first())
634         while query.next():
635
636             temp_value = ((query.value(0)))
637
638     #model = QSqlQueryModel()
639
640     if( not query.first()):
641         logger.info('this ingredient %s is missing' % self.panda_ingredients[k][2])
642         self.missing_ingredient = self.panda_ingredients[k][2]
643         self.MyMissingIngredients()
644     else:
645         #fill variables
646         energy_v.append(float(query.value(energy)))
647         protein_v.append(float(query.value(protein)))
648         carbohydrate_v.append(float(query.value(carbohydrate)))
649         fat_v.append(float(query.value(fat)))
650         temp_weight = float(self.RecipeModel.recipes[k][0])/100. # all is normalized to per 100 g
651
652         Total_calories_per_gm = Total_calories_per_gm+temp_weight*energy_v[k]
653         Total_protein_per_gm = Total_protein_per_gm + temp_weight*protein_v[k]
654         Total_fat_per_gm = Total_fat_per_gm + temp_weight*fat_v[k]
655         Total_carbs_per_gram = Total_carbs_per_gram + temp_weight*carbohydrate_v[k]
656
657         Total_weight = Total_weight+temp_weight*100.
658
659         print(self.RecipeModel.recipes[k][0],', ', Total_calories_per_gm)
660         logger.debug('ingredient: %s' % self.panda_ingredients[k][2])
661         logger.debug('energy: %f' % energy_v[k])
662         logger.debug('protein: %f' % protein_v[k])
663         logger.debug('carbohydrate: %f' % carbohydrate_v[k])
664         logger.debug('fat: %f' % fat_v[k])
665     if(Total_weight == 0.):
666         return
667     temp_cal = Total_calories_per_gm/Total_weight
668     temp_fat = Total_fat_per_gm/Total_weight
669     temp_prot = Total_protein_per_gm/Total_weight
670     temp_carb = Total_carbs_per_gram/Total_weight
671     logger.debug('total calories per gram: %f' % temp_cal)
672     logger.debug('total carbs per gram: %f' % temp_carb)
673     logger.debug('total protein per gram: %f' % temp_prot)
674     logger.debug('total fat per gram: %f' % temp_fat)
675
676     # convert to string
677     string_cal = f"{temp_cal:.2f}"
678     string_fat = f"{temp_fat:.2f}"
679     string_prot = f"{temp_prot:.2f}"
680     string_carb = f"{temp_carb:.2f}"
681
682
683     #Update the values on the form

```

Feb 23, 2025 13:05

ControlDB_new.py

Page 13/20

```

684         self.MRD.cal_line.setText(string_cal)
685         self.MRD.fat_line.setText(string_fat)
686         self.MRD.prot_line.setText(string_prot)
687         self.MRD.carb_line.setText(string_carb)
688
689         # put values onto form
690
691
692
693         self.new_recipe_record=(self.new_recipe_name,string_cal,string_prot,string_carb,string_fat,self.pack_record(self.panda_ingr
edients))
694         return
695
696     def add_row(self):
697         """ adds row to reipe table"""
698         row_position = self.RecipeModel.rowCount()
699         self.RecipeModel.insertRows(row_position)
700
701         return
702
703     def on_item_click(self, index):
704
705         #columns and rows start from 0
706         item_text = index.data()
707         item_row = index.row()
708         item_column = index.column()
709         return
710
711
712         #print(f"Clicked item: {item_text}")
713         #print(index.row())
714
715     def save_recipe(self):
716         """gets called from clicking the save button"""
717         record = self.RecipeModel.StoreValue()
718         logger.info("ingredients %s" % record)
719         #create original string
720         record = self.pack_record(record)
721         # Now update the record
722
723         #check if this is a new recipe or an updated one
724         if self.new_recipe_name != None:
725             rec_name=self.new_recipe_name
726             self.create_pandas_frame(name=rec_name)
727
728             self.insert_record()
729         else:
730             rec_name = self.selected_recipe
731             self.create_pandas_frame(name=rec_name)
732
733             self.update_record('recipes', 'ingredients', record, rec_name)
734
735         #self.new_recipe_name = None # reset recipe name
736         #here we call create_pandas_frame tframe for CalculateCalories start producing the
737
738         #remove window
739         #self.MRD.close()

```

Feb 23, 2025 13:05

ControlDB_new.py

Page 14/20

```

740
741
742     return
743     def insert_record(self):
744         """inserts new recipe into table
745         columnname is a list of all the columns
746         record is a corresponding list"""
747         # create sql statement
748
749         record =self.new_recipe_record
750
751         # since ID is a non-zero number we need to find the highest number and then add one for the next reipce
752         sql = 'SELECT id from recipes;'
753         ID_number=[]
754         myquery = self.do_sql(sql)
755         while myquery.next():
756
757             ID_number.append(int(myquery.value(0)))
758             #find the highest numer in the list
759             imax = max(ID_number)
760             new_recipe_id = str(imax+1)
761
762
763             b= new_recipe_id+',\'+record[0]+'\'\''+record[1]+'\',\''+record[2]+'\',\''+record[3]+'\',\''+record[4]+'\',\''+record[5]+'\'\'
764
765             sql = 'INSERT INTO recipes (id,name,energy,protein,carbohydrate,fat,ingredients) VALUES ('+b+')';
766             logger.debug("insert recipe %s" % sql)
767             #finally excecute the sql
768             self.do_sql(sql)
769             return
770
771
772
773
774
775         #self.new_recipe_record=(self.new_recipe_name,string_cal,string_prot,string_carb,string_fat,self.pack_record(self.panda_ing
776         redients))
777
778         # the sql statements should be
779         # sql = INSERT INTO table1 (name,energy,protein,carbohydrate,fat,ingredients) VALUES (row_name,record[0],record[1],record[2]
780         record[3],record[4]);
781         logger.debug(" insert statement %s % sql")
782         self.do_sql(sql)
783
784         return
785
786
787     def update_record(self,table,column_name,record,row_name):
788         """ updates record for name in table
789         example SQL statement
790         UPDATE recipes SET ingredients = '30 g almonds 60 g dry_tomatoes 517 g tenderloin' WHERE name='basil_tenderloin';
791         """
792
793         sql = 'UPDATE '+table+' SET '+column_name+' =\'+record+'\' WHERE name =\'+row_name+'\'';
794         self.do_sql(sql)
795
796         return

```

Feb 23, 2025 13:05

ControlDB_new.py

Page 15/20

```

795
796 def pack_record(self, record):
797     """ put record back into original form for ingredients, which is just one string """
798     b=[]
799     c=''
800     #self.panda_ingredients are the correct list for creating the panda frame
801     self.panda_ingredients = a =record
802     for k in range(len(a)):
803         if (a[k][0]!=''): break
804         b.append(a[k][0]+' '+a[k][1]+' '+a[k][2]+' ')
805
806     for k in range(0,len(b)):
807         c = c+ b[k]
808
809     # remove last space
810     d = c[0:len(c)-1]
811     return(d)
812
813
814 def getSelectedText(self,s):
815     self.selected_text = s
816     logger.info("You have selected %s text" % self.selected_text)
817     return
818
819
820 def getSelectedRecipe(self,s):
821     #Clear Buffer of recipes
822
823
824     self.RecipeModel.recipes = []
825
826     self.selected_recipe = s
827     logger.info("You have selected %s recipe" % self.selected_recipe)
828
829     # now we populate the listview
830     self.FillRecipeIngredientList()
831
832     # Now fill qlist
833     self.DisplayRecipeList()
834
835     # send signal list has been updated
836     self.RecipeModel.layoutChanged.emit()
837
838
839
840     return
841
842 def do_sql(self,sql):
843     """executes a sql statement"""
844     query = QSqlQuery(sql,db=self.mycal_db)
845     model = QSqlQueryModel()
846     logger.info("query in do_sql %s" % sql)
847     model.setQuery(query)
848     if query.lastError().isValid():
849         logger.error(f"Query error: {query.lastError().text()}")
850         # if this is a inserty error try replace.
851

```

Feb 23, 2025 13:05

ControlDB_new.py

Page 16/20

```

852
853
854     return query
855
856
857 def FillRecipeIngredientList(self):
858     # here we get the ingredients from the Recipe database
859     # this is done with a query
860
861
862
863     sql = ' SELECT ingredients from recipes WHERE recipes.name LIKE \''+self.selected_recipe + '\';'
864     query = QSqlQuery(sql,db=self.mycal_db)
865     model = QSqlQueryModel()
866
867     model.setQuery(query)
868     while query.next():
869
870         temp_value = ((query.value(0)))
871         # now we have to work with string
872         # the old database still had *u000a in front of the number
873         # need first to strip this
874         #check if temp contains *u000a
875         mask = 'u000a'
876         if(mask in temp_value):
877             temp_new = self.StripUnicode(temp_value,mask)
878             #temp_new = self.StripUnicode(temp_new,'*')
879
880
881
882
883     else:
884         #already the new entry way
885         temp_new1 = []
886         temp_new = temp_value.split(' ')
887         for k in range(0,len(temp_new),3):
888             temp_new1.append([temp_new[k],temp_new[k+1],temp_new[k+2]])
889         self.recipe_ingredients = temp_new1
890         return
891
892
893     delimiter = "g"
894     result = temp_new.replace('u000a','')
895
896
897     result1 = self.split_and_keep(result, delimiter)
898     # strip last *
899     result1l= result1[0]
900     result2 = result1l[:len(result1[0])-1].split('*')
901     # we know that a string with underscores has a max of three spaces. Any number larger than this means words with spaces.
902     #in other words 4 spaces means the 3rd has to be replaced with _ 5 spaces mean the 3rd and 4th
903     result3 =[]
904     for a in result2:
905         space_indices = []
906         for i, char in enumerate(a):
907             if char == ' ':
908                 space_indices.append(i)

```


Feb 23, 2025 13:05

ControlDB_new.py

Page 17/20

```

909
910         if len(space_indices) == 4:
911             na = a[:space_indices[2]] + '_' + a[space_indices[2] + 1:]
912
913             result3.append(na)
914
915         elif len(space_indices) == 5:
916             na = a[:space_indices[2]] + '_' + a[space_indices[2]+1:space_indices[3]] + '_' + a[space_indices[3] + 1:]
917             result3.append(na)
918
919         else:
920             result3.append(a)
921
922
923     recipe_string = result3
924     #finally split every field up into 3 '10 g bean' -> [10,'g','bean']
925     result4 = []
926     for i in range(len(result3)):
927         temp = (result3[i].split(' '))
928         result4.append(temp[0:3])
929     self.recipe_ingredients = result4
930     return
931
932 def DisplayRecipeList(self):
933     """loop through ingredient list and add"""
934
935
936     for k in range(len(self.recipe_ingredients)):
937         print(self.recipe_ingredients[k])
938
939         self.RecipeModel.recipes.append(self.recipe_ingredients[k])
940     return
941
942
943 def split_and_keep(self,text, delimiter):
944     """deals with spaces instead of underscores"""
945     result = []
946     temp = ""
947     for char in text:
948         if char == delimiter:
949             result.append(temp + char)
950             temp = ""
951         else:
952             temp += char
953     if temp:
954         result.append(temp)
955     return result
956
957
958
959
960 def StripUnicode(self,mystring,mask):
961     newstring =mystring.replace(mask,'')
962     return newstring
963
964
965 def GetRecipeList(self):

```

Feb 23, 2025 13:05

ControlDB_new.py

Page 18/20

```

966         """sql query to get all the names of the recipies
967         will be used to populate the recipe qlistbox"""
968         sql = 'SELECT name from recipes ; '
969         query = QSqlQuery(sql,db=self.mycal_db)
970         model = QSqlQueryModel()
971
972         model.setQuery(query)
973         recipe_list = []
974         while query.next():
975             recipe_list.append(str(query.value(0)))
976
977
978         # now sort them alphabetically
979         items = [recipe_list[i] for i in range(len(recipe_list))]
980         items.sort(key=str.lower) #this way no diff between upper and lower case
981
982         return (items)
983
984
985
986
987
988     def ModifyRecipe(self):
989         pass
990
991
992     def SizeWindow(self,window,myposit,mysize):
993         """resizes the specified window ,where the parameters are two integer lists"""
994         window.setGeometry(myposit[0],myposit[1],mysize[0],mysize[1])
995         return
996
997     def calculate_portion_calory(self,myrecipe=None,myportion=0.):
998         """ this calucaltes the calories of a portion for  given recipe,
999         where the portion size is given in grams."""
1000
1001         # sql statement: SELECT enegry FROM recipes WHERE NAME = 'raspberry_tart';
1002         sql = 'SELECT energy FROM recipes WHERE NAME=\'' + myrecipe + '\'';
1003         query = self.do_sql(sql)
1004         while query.next():
1005
1006             cal_per_gram = ((query.value(0)))
1007             portion_calory = float(cal_per_gram)*myportion
1008             logger.info("your total calories are %.1f " % portion_calory)
1009             return portion_calory
1010
1011
1012
1013
1014
1015 #####
1016 # section for signals and slots
1017
1018
1019     def Cancel(self,window):
1020         """ closes window without anything"""
1021         window.close()
1022

```

Feb 23, 2025 13:05

ControlDB_new.py

Page 19/20

```

1023
1024
1025
1026 def SaveIngredients(self):
1027     """saving the ingredients"""
1028     if( not self.swiss_foods):
1029         Ing_name      = self.name_label.text()
1030         Ing_calory    = float(self.calory_label.text())
1031         Ing_carb      = float(self.carb_label.text())
1032         Ing_fat       = float(self.fat_label.text())
1033         Ing_prot      = float(self.prot_label.text())
1034     else:
1035         Ing_name      = self.InTa_name
1036         Ing_calory    = self.InTa_energy
1037         Ing_carb      = self.InTa_carbohydrate
1038         Ing_fat       = self.InTa_fat
1039         Ing_prot      = self.InTa_protein
1040
1041
1042     # this is the record we got from the form input
1043
1044     #Check all fields are filled out
1045     if(Ing_calory !="" and Ing_carb !="" and Ing_fat!="" and Ing_prot != ""):
1046         self.Inform.destroy() # destrorys the window.
1047         #self.Inform.close() #
1048
1049     #now we need to add the values to the ingredients table
1050     # first load table
1051
1052
1053     self.ViewTable(self.CM.ingred_table,suppress_columns=self.ingredients_suppress_columns)
1054     self.model = QSqlQueryModel()
1055     self.table_view.setModel(self.model)
1056     #next we check we don't have an entry yet
1057
1058
1059     #sql = "SELECT name from ingredients WHERE ingredients.name LIKE 'Almond%' ; "
1060     sql = 'SELECT name from ingredients WHERE '+self.CM.ingred_table+'.name LIKE '+"'"+Ing_name+"'"'; '
1061     query = QSqlQuery(sql,db=self.mycal_db)
1062
1063     self.model.setQuery(query)
1064     while query.next():
1065         logger.error(' entry name already exists, try again %s' % query.value(0))
1066         self.CreateIngredientsForm1()
1067         #myvalue = query.value(0)
1068     # Now we need to add this ingredient to the table
1069
1070     self.record = [Ing_name,Ing_calory,Ing_carb,Ing_fat,Ing_prot]
1071     self.InsertRecord(table = self.CM.ingred_table)
1072     return
1073
1074
1075 if __name__ == '__main__':
1076     app = QApplication(sys.argv)
1077
1078     Title=None
1079     db_name='recipe_ak'

```

Feb 23, 2025 13:05

ControlDB_new.py

Page 20/20

```
1080 db_user='klein'
1081 db_system='QPSQL'
1082 config_file = '/Users/klein/git/qt_exercises/config/config_mycal.json'
1083
1084 with open('/Users/klein/git/qt_exercises/config/pw.txt', 'r') as file:
1085     db_pwd = file.read().rstrip()
1086
1087
1088
1089
1090 suppress_columns = ['Sugar',
1091                     'Portions',
1092                     'Description',
1093                     'Saturated_fat',
1094                     'Fiber',
1095                     'Salt',
1096                     'Sodium',
1097                     'Ingredients',
1098                     'Time',
1099                     'Images',
1100                     'vegetarian']
1101 columnwidth = [40,200,200,200,200,200]
1102
1103 window = ControlDB_new(Title = "ControlDB",
1104                        db_name=db_name,
1105                        db_user=db_user,
1106                        db_system=db_system)
1107 #window.SetSize(800,500)
1108 #window.SetPosition(100,500)
1109
1110 window.setStyleSheet("background-color: white;")
1111 #window.CreateCalendar()
1112
1113
1114
1115 #window.show()
1116 window.ConnectDataBase()
1117 window.ShowTables()
1118 #window.calculate_portion_calory(myrecipe = 'raspberry_tart',myportion=200.)
1119 #window.ViewTable('recipes', suppress_columns=suppress_columns, columnwidth=columnwidth)
1120
1121 window.MyRecipes_new()
1122 #window.CreateIngredientsForm1()
1123
1124 # now run the app
1125 app.exec()
```